

Project - Final Report

The signed declaration that the work you're submitting is your own original work

The work being submitted is my own original work.

A high level description of your program

Planthony's Last Stand is a real time Space Invaders-esque game, where the player takes control of Planthony who must defend himself from a never ending horde of blue devils. The player must shoot at the devils using Planthony's peapod blaster to knock those devils down!

The gameplay consists of moving Planthony left and right (pressing 'a' or 'd') at the bottom of a 4x8 grid to aim at and shoot (pressing spacebar) the devils making their way down to the bottom of the game screen. If the player is tired from the non-stop action, they can pause the game (pressing 'p') and can continue whenever they feel like by simply unpausing (pressing 'p' again). Killing devils increases the score, which is printed to the player in the Mars Run I/O terminal.

The game starts out easy, with devils moving and spawning slowly from the top, but every 30 seconds the game's difficulty will ramp up! Should even a single devil make it to Planthony, that spells game over! How long can you help Planthony last...

The game makes use of both the Keyboard and Display MMIO and Bitmap Display (set unit width - 2, unit height - 2, disp. width - 256, disp. height - 512) to take real time inputs from the player and render the game. The game also has sound effects and music, so have your volume on!

A list of the core functions and operations of your project

[macros - some include subroutines that get called]

push (%x) - to stack

pop (%x) - from stack

Sleep (%x) - for %x ms

newline - for terminal gameplay

exit - for testing

file (%file_name, %file_size, %pixel_width, %pixel_height) - open and read file to memory, just made sprite management WAY easier

file_M#

file_frameBuffer_M#

file_width_M#

file_height_M#

songLoop (%song_notes, %song_length, %song_instrument, %song_volume) - keep track of place in song, just made song management WAY easier

jal song_M#

[functions]

makeSpace - initialize gridSpace

checkKeyboard - read MMIO input, update player position, shoot bullet, etc. basically the main hub for player interaction with *many* subroutines involved

updateState - change gameState for things outside of the player's direct controller (occur without input)

initialTime - this is the time implementation, get a starting time to compare against

currentTime - this is the time implementation, increases difficulty every 30s to a cap

moveBullet - update bullet position and check for collision

moveEnemy - update enemy position and check for collision

spawnEnemy - creates an enemy in a "random" column

print - this was used during development, prints to terminal gridSpace, can be used instead if computer can't handle printBitmap

printBitmap - this takes the gridSpace where all the logic took place, and prints it to the bitmap display with the appropriate sprite; I used a div instruction on a single index because of how the remainder and quotient act as the row and column with appropriate shifting (this method works better as the index is constantly needed to feed the correct data), it calls...

printSprite - the reason why I chose to use a macro for sprite management, this function acts as a sort of stamp, where I can give simply pass the location to draw (a base address) and data all linked to the same sprite (file_ " _M#) and it will place it on the bitmap very easily; it can easily work as a standalone function in any future project where bitmap displaying is involved

clearDead - whenever a collision occurs, a dead enemy is created and the address of the enemy gets saved onto stack, this clears all dead enemies off of stack

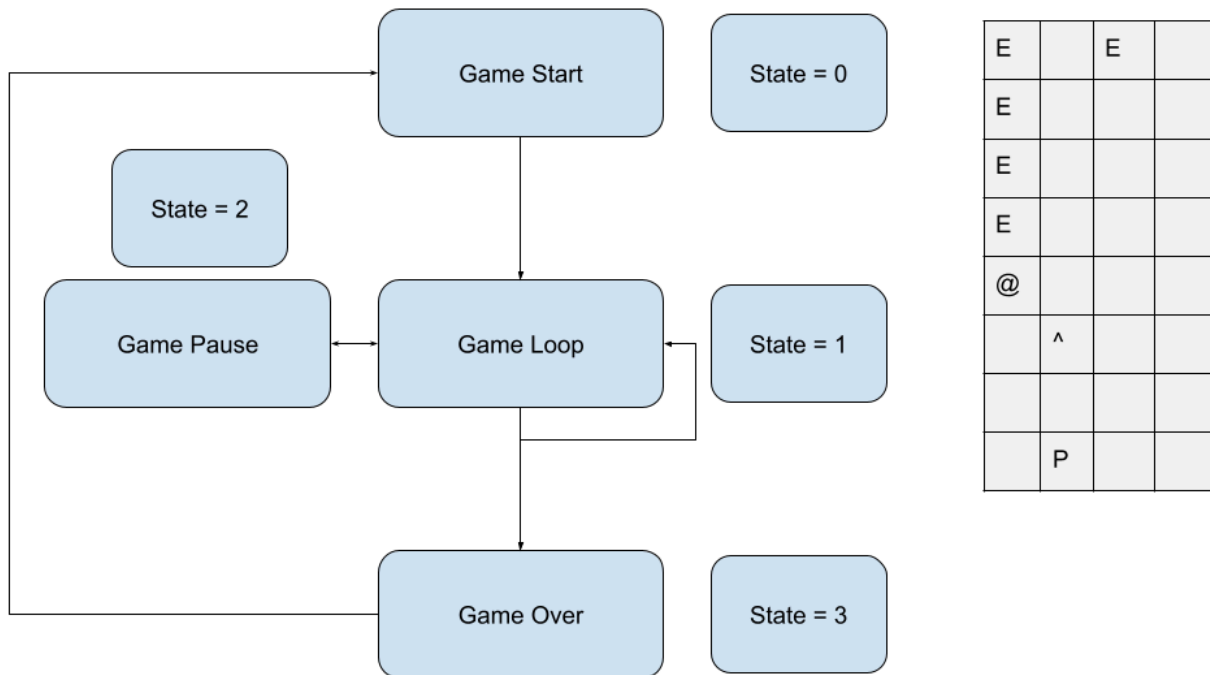
lfsr32 - this is the randomization feature for enemy spawns, maybe calling it every frame was overkill...

gameLoop - this is the main gameplay of my program, it runs nearly all of the above subroutines, in a specific order (*elaborated in the high level description of the code*)

calculateArrayAddress - this gets called often, it just helped with the 2D char array (gridSpace)

sound_situation - there are multiple of these function, easier to manage when they are separate subroutines that can get called

A high level description of the code, written in C or pseudo code or flow chart format



Game Start

Initialize all songs and files (will only occur once, already stored in memory)
 Reset all registers
 Initialize Ifsr32 with seed
 Reset global variables
 Initialize gridSpace
 Reset song indices

Repeat until player input(Play note of song1)
 After player input

GameLoop:

Play note of song2
 Move bullets
 Check the keyboard -> Move player/shoot bullet/pause game (can only do one at a time)
 Check if time since GameLoop beginning is more than 30 seconds and if so, increase difficulty
 Get turn index, if (turn index mod difficulty is zero), do(move enemy, spawn enemy)
 Increase turn index
 Print current grid to bitmap

Remove dead enemies
Get a new random number
Check and change game state
Loop back to GameLoop

GamePause

This state occurs when the player presses p in check the keyboard
Stay here until player presses p again

GameOver

When the player takes a hit from an enemy (enemy got to the bottom)
Play note of song3
Repeat until player presses any key -> go to GameStart

Summary of the tests you've performed on your code and examples of the test results

Spamming Inputs

Pressing keys on the pause screen

The game properly switches between game states and loads the correct sprites.

There isn't much variation in terms of what the player is capable of doing, since it uses a set seed, the game should generally run the same. Since I use time, and the game is in real time there may be some variation caused by how fast or slow the game runs on specific computers etc. This isn't an issue I found to be necessary to resolve.

The music is very choppy during the main gameplay loop, which I wasn't able to debug.

The results should include screen shots, or links to audio/video recordings on Youtube or shared Google files, as appropriate

I ran this program for 3 minutes and 30 seconds, recording the result. In this test run I tested the title screen, gameplay, game over, playing again, and pausing.

Here is a link to the test run which is an unlisted video with game audio that I recorded.

https://youtu.be/GNj9-gUz7rc?si=xixaNwUqtH_X2gxQ

The description of the video timestamps when the game switches states.

Summary of all tasks that were completed by the time the final report was due

The minimum accomplished:

Conditional branches

Nested loops

Use of 2-D arrays

Use of the stack to store the \$ra register

Extra features:

Sound

Music

Randomized Enemies using a 32 bit lfsr

Pause screen

Bitmap Display

Memory Mapped Input Output

Loading files

Time

List of tasks that could not be completed by the due date

Scoring based on rhythm

Ranking system based on score

Multiple lives displayed on the bitmap

Score displayed on the bitmap

Conclusion, stating what you'd do differently if you had it to do over again

I definitely would start with using new files to test and debug features. A lot of time was spent trying to fix code inside of my main program, and having to scroll back and forth a huge file. By isolating the code I wanted to test (with dependencies such as push and pop for instance), I could easily look at the registers without having to spend even more time with breakpoints within hundreds of lines of code. Once the code was working as intended, I could simply copy and paste the updated code into my main program.

Estimated total time to complete the project, in hours.

70 hours

At the end of the draft report, you must include ALL the code with comments (formatted in the style of the MIPS code samples) you've written to date, pasted into the draft report file, plus:

```
# Author:      Rhilo Sotto
# Description: Project in MIPS

# open bitmap display and keyboard mmio simulator
# unit width - 2, unit height - 2, disp. width - 256, disp. height - 512

# https://convertio.co/rgba-converter/
# to turn png files into usable rgba

# all art provided by Danelle Verdadero, huge thanks!

# CONSTANTS
    .eqv  X_SPACE      4
    .eqv  Y_SPACE      8
    .eqv  FB_WIDTH     0x20000      # disp. width * disp. length
    .eqv  FB_LENGTH    0x100 # disp. width
    .eqv  START_DIFFICULTY 4
    #.eqv

# MACROS

# stack push
    .macro push (%x)
        addi  $sp, $sp, -4      # make space on stack
        sw    %x, 0($sp)      # store onto stack
    .end_macro

# stack pop
    .macro pop (%x)
        lw    %x, 0($sp)      # copy value from stack
```

```

        addi    $sp, $sp, 4    # return space to stack
.end_macro

# sleep for 250 ms
.macro sleep    (%x)
    li        $a0, %x        # delay in ms between loops, 250 ms
    li        $v0, 32
    syscall
.end_macro

# print a new line
.macro newline
    la        $a0, newLine
    li        $v0, 4
    syscall
.end_macro

# exit the program
.macro exit
    li        $v0, 10
    syscall
.end_macro

# open and store (.rgba) file into memory
.macro file    (%file_name, %file_size, %pixel_width, %pixel_height)
.data
# la file_frameBuffer_M#
# to access data during runtime
    .align    2    # align address onto word boundary
    file_frameBuffer:    .space %file_size    # size of file in bytes (hex)
    file_width:    .word    %pixel_width    # width in pixels
    file_height:    .word    %pixel_height    # height in pixels
    .align    2    # align onto word boundary
    file_name:    .asciiz %file_name    # store name of file

.text
# this part only needs to occur during initialization, similar to songLoop macro
    file_open:
    # open file
        li        $v0, 13
        la        $a0, file_name
        li        $a1, 0    # read-only flag
        li        $a2, 0    # mode is ignored
        syscall # $v0 now contains the file descriptor

```

```

file_read:
# read file into memory
    add    $a0, $v0, $zero    # move file descriptor to $a0
    li     $v0, 14
    la     $a1, file_frameBuffer
    li     $a2, %file_size
    syscall

# due to endian-ness, the colors are different from the rgba file
# attempting to fix this during runtime resulted in
# a massive increase in startup time, so while it is possible
# it was SUPER SLOW and isn't in this project

file_close:
# close file
    li     $v0, 16
    syscall

j         file_initializeExit
file_exit:
jr        $ra
file_initializeExit:
.end_macro

# loop through a song
.macro songLoop (%song_notes, %song_length, %song_instrument, %song_volume)
.data
    song_index: .byte 0    # starts at 0, will loop back
.text
j         song_initializeExit
# push/pop $ra before/after jal
# jal song_M#
song:
    la     $t0, %song_notes # load song address
    lb     $t1, song_index   # load spot in song
    sb     $t1, song_index
    li     $t2, %song_length # load length of song

    add    $t0, $t0, $t1    # get current note address
    addi   $t1, $t1, 1      # ++index

    bge    $t1, $t2, song_reset # reset song index? (index < length)

```

```

j      song_play
song_reset:
    li    $t1, 0 # index = 0
song_play:
    sb    $t1, song_index

    lb    $a0, 0($t0) # this is the pitch
    beq   $a0, -1, song_exit # if the note is -1, don't play
    lb    $t3, 1($t0) # check if next note is -1 (extend note)
    beq   $t3, -1, song_extend
    li    $a1, 250 # this is the duration in ms
    j     song_noExtend
song_extend:
    li    $a1, 500 # double duration
song_noExtend:
    li    $a2, %song_instrument # this is the instrument
    li    $a3, %song_volume # this is the volume
    li    $v0, 31 # this is the syscall service number
    syscall
song_exit:
    jr    $ra
song_initializeExit:
.end_macro

# the data section
.data
    # bitmap display base address
    framebuffer: .space 0x20000 # entire bitmap display
    framebufferWidth: .word 0x80

    gameScreenLocation: .word 0x1020 # location to start drawing gameplay

    score: .byte 0 # score
    #scoreLocation: .word 0x0 # location on bitmap display to update score
    #scoreWidth: .word 0x0 # something

    #lives: .byte 3 # lives
    #livesLocation: .word 0x0 # location on bitmap to update lives
    #livesWidth: .word 0x0 # something

    # GLOBAL VARIABLES
    gameState: .byte 0 # game state
    playerInput: .word 0 # player input
    playerHP: .byte 1 # player HP

```

```

playerXpos: .byte 0      # player X position
difficulty: .byte START_DIFFICULTY # difficulty (lower = harder)
turnIndex:  .word 0      # how many turns (game loops) have happened
deadCount:  .byte 0      # enemies killed, used when clearing dead enemies to
know how many times to pop off stack

```

```

startTimeUpper: .word 0      # update to when game loop begins
startTimeLower: .word 0

```

```

# SONG DATA
# song_M0
song1: .byte 59, 64, 66, 69, 66, 64, 59, 64, 66, 69, 66, 64, 59, 64, 66, 69, 66, 64, 61,
62, 61, 57
# 22 notes
# song_M1
# [BAD APPLE]
song2: .byte 63, 65, 66, 68, 70, -1, 75, 73, 70, -1, 63, -1, 70, 68, 66, 65, 63, 65, 66, 68,
70, -1, 68, 66, 65, 63, 65, 66, 65, 63, 62, 65, 63, 65, 66, 68, 70, -1, 75, 73, 70, -1, 63, -1, 70, 68,
66, 65, 63, 65, 66, 68, 70, -1, 68, 66, 65, -1, 66, -1, 68, -1, 70, -1
# 64 notes
# song_M2
song3: .byte 62, 74, 57, 69, 55, 67, 53, 65, 50, 62, 53, 65, 55, 67, 57, 69
# 16 notes

```

```

# 2D Array
gridSpace: .space 32      # X_SPACE * Y_SPACE = 5 * 10 = 50
# representation in array (technically arbitrary w/printBitmap)
empty: .byte ''
enemy:  .byte 'V'
player: .byte 'P'
bullet: .byte '^'
dead:   .byte 'X'

```

```

newLine: .byte '\n'

```

```

# lfsr32
seed: .word 1437269760 # initial seed value 0x55A AFF00
lfsr: .word 0x0        # memory location of lfsr

```

```

.text
# song_M0
songLoop (song1, 22, 54, 50)
# song_M1

```

```
songLoop    (song2, 64, 54, 50)
# song_M2
songLoop    (song3, 16, 54, 50)
```

```
# sprites, only need to load once
# file_M3
file    ("player16.rgb", 0x400, 0x10, 0x10)
# file_M4
file    ("bullet16.rgb", 0x400, 0x10, 0x10)
# file_M5
file    ("enemy16.rgb", 0x400, 0x10, 0x10)
# file_M6
file    ("dead16.rgb", 0x400, 0x10, 0x10)
# file_M7
file    ("blank16.rgb", 0x400, 0x10, 0x10)
# file_M8
file    ("game16.rgb", 0x20000, 0x80, 0x100)
# file_M9
file    ("title16.rgb", 0x20000, 0x80, 0x100)
# file_M10
file    ("gameover16.rgb", 0x20000, 0x80, 0x100)
# file_M11
file    ("pause16.rgb", 0x20000, 0x80, 0x100)
```

initialize:

```
# reset all registers
li    $v0, 0
li    $v1, 0

li    $a0, 0
li    $a1, 0
li    $a2, 0
li    $a3, 0

li    $t0, 0
li    $t1, 0
li    $t2, 0
li    $t3, 0
li    $t4, 0
li    $t5, 0
li    $t6, 0
li    $t7, 0
```

```

li    $s0, 0
li    $s1, 0
li    $s2, 0
li    $s3, 0
li    $s4, 0
li    $s5, 0
li    $s6, 0
li    $s7, 0

lw    $a0, seed    # initialize lfsr32 with seed
jal    lfsr32
li    $a0, 0 # always have $a0 as 0 when calling lfsr3

```

reset global variables

```

li    $t0, 0 # reset to 0
sb    $t0, gameState
sw    $t0, playerInput
sb    $t0, score
sb    $t0, playerXpos
sw    $t0, turnIndex

```

```

li    $t0, 1 # reset to 1
sb    $t0, playerHP

```

```

li    $t0, START_DIFFICULTY    # reset to constant
sb    $t0, difficulty

```

```

li    $t0, 0

```

Initialization of gridSpace

t0 - index i, \$t1 - index j, \$t2 - gridSpace address, \$t3 - default grid value "empty"

makeSpace:

```

lb    $t3, empty

```

```

li    $t0, 0 # i = 0

```

makeSpace_whileX:

```

bge    $t0, X_SPACE, makeSpace_doneX # while (i < X_SPACE)

```

```

li    $t1, 0 # j = 0

```

makeSpace_whileY:

```

bge    $t1, Y_SPACE, makeSpace_doneY # while (j < Y_SPACE)

```

initialize gridSpace with '.'

```

        la    $a0, gridSize
        add   $a1, $zero, $t0      # row iterates after column, row-major
        li    $a2, Y_SPACE
        add   $a3, $zero, $t1
        jal   calculateArrayAddress # calls calculateArrayAddress
subroutine
        add   $t2, $zero, $v0

        sb    $t3, 0($t2)

        # initialize gridSize with '.'

        addi   $t1, $t1, 1    # ++j
        j      makeSpace_whileY
makeSpace_doneY:

        addi   $t0, $t0, 1    # ++i
        j      makeSpace_whileX
makeSpace_doneX:
makeSpace_exit:

        lb     $t0, player
        sb     $t0, gridSize + 28

        # draw title screen
        la     $a0, frameBuffer
        la     $a1, file_frameBuffer_M9
        lw     $a2, file_width_M9
        lw     $a3, file_height_M9
        jal    printSprite

reset_songIndex:    # reset song index
        li     $t0, 0
        sb     $t0, song_index_M0
        sb     $t0, song_index_M1
        sb     $t0, song_index_M2

main:

gameTitle:
        sleep  (250)
        jal    song_M0

        jal    checkKeyboard

```

```

        jal    updateState

        j      gameTitle

gameLoop:
    sleep    (250)
    jal      song_M1

    playerTurn:
        jal    moveBullet    # move bullets
        jal    checkKeyboard    # player input - either move or shoot bullet

        jal    currentTime    # checks if time elapsed is greater than 30s, increases difficulty

        lw     $t0, turnIndex
        lb     $t1, difficulty
        rem    $t2, $t0, $t1    # check if (turnIndex % difficulty) == 0
        bne    $t2, 0, afterEnemyTurn    # give enemy turn every 6>5>4>3>2

        enemyTurn:
            jal    moveEnemy    # move enemies
            jal    spawnEnemy    # create an enemy
        afterEnemyTurn:

            lw     $t0, turnIndex
            addi    $t0, $t0, 1
            sw     $t0, turnIndex # ++turnIndex

            #jal    print    # replace with bitmap print
            jal    printBitmap    # make more advanced

            jal    clearDead    # dead get printed above, and get removed after print

            li     $a0, 0
            jal    lfsr32    # get new random number

            jal    updateState    # check and change game state

            j      gameLoop    # loop

gamePause:
    sleep    (250)

```

```
jal    checkKeyboard
jal    updateState
```

```
j      gamePause
```

gameOver:

```
sleep  (250)
jal    song_M2

jal    checkKeyboard
jal    updateState

j      gameOver
```

updateState:

```
# SWITCH CASE
lb     $t0, gameState
beq    $t0, 0, updateState_title
beq    $t0, 1, updateState_main
beq    $t0, 2, updateState_pause
beq    $t0, 3, updateState_over2

j      updateState_exit
```

updateState_title:

```
li     $t0, 0 # set to game state 0 (title)
sb     $t0, gameState
j      updateState_exit
```

updateState_main:

```
li     $t0, 1 # set to game state 1 (loop)
sb     $t0, gameState

lb     $t0, playerHP
ble    $t0, 0, updateState_over    # player hp >= 0

j      updateState_exit
```

updateState_pause:

```
li     $t0, 2 # set to game state 2 (paused)
sb     $t0, gameState

j      updateState_exit
```

```

updateState_over:
    lb      $a0, score
    li      $v0, 1
    syscall # print score to terminal #BITMAP
    newline

    # print gameOver screen
    la      $a0, framebuffer
    la      $a1, file_frameBuffer_M10
    lw      $a2, file_width_M10
    lw      $a3, file_height_M10
    jal     printSprite

    jal     sound_gameOver

    la      $ra, gameOver
updateState_over2:
    li      $t0, 3 # set to game state 3 (game over)
    sb      $t0, gameState
    j       updateState_exit

updateState_exit:
    jr      $ra

```

```

checkKeyboard:
    lw      $t1, 0xffff0000 # check if new key is pressed
    sw      $t1, playerInput
    beq     $t1, 0, checkKeyboard_exit # if (any new key)
    lw      $t1, 0xffff0004 # load keyboard input
    sw      $t1, playerInput

    # CHECK GAMESTATE
    lb      $t0, gameState
    beq     $t0, 0, checkKeyboard_title
    beq     $t0, 1, checkKeyboard_main
    beq     $t0, 2, checkKeyboard_pause
    beq     $t0, 3, checkKeyboard_over
checkKeyboard_title:
    li      $t0, 1 # set to game state 1 (loop)
    sb      $t0, gameState
    # draw gameplay screen
    la      $a0, framebuffer

```

```

la    $a1, file_frameBuffer_M8
lw    $a2, file_width_M8
lw    $a3, file_height_M8
jal   printSprite

jal   sound_titleToGamePlay

jal   initialTime

j     gameLoop

```

checkKeyboard_main:

```

# CHECK KEY PRESSED
beq   $t1, 97, movePlayer_left    # case 'a'
beq   $t1, 100, movePlayer_right  # case 'd'
beq   $t1, 32, shootPlayer        # case ' '
beq   $t1, 112, pauseGame         # case 'p' - pause

```

```

j     checkKeyboard_exit

```

checkKeyboard_pause:

```

bne   $t1, 112, checkKeyboard_exit    # case 'p' - unpause
li    $t0, 1    # set to game state 1 (loop)
sb    $t0, gameState

```

```

# draw gameplay screen

```

```

la    $a0, frameBuffer
la    $a1, file_frameBuffer_M8
lw    $a2, file_width_M8
lw    $a3, file_height_M8
jal   printSprite

```

```

j     gameLoop

```

checkKeyboard_over:

```

# check if quitting
bne   $t1, 113, notQuitting    # player pressed 'q'
exit   # end program
notQuitting:
li    $t0, 0    # set to game state 0 (title)
sb    $t0, gameState
# check if quitting

```

```

j     initialize

```

checkKeyboard_exit:

jr \$ra

movePlayer:

movePlayer_right:

li \$t2, 1

li \$t3, 3 # X_SPACE - 1 = 3

lb \$t4, playerXpos

bge \$t4, \$t3, movePlayer_exit # if (xpos < X_SPACE - 1)

j movePlayer_move

movePlayer_left:

li \$t2, -1

lb \$t4, playerXpos

ble \$t4, 0, movePlayer_exit # if (xpos > 0)

j movePlayer_move

movePlayer_move:

push (\$ra)

la \$a0, gridSpace

li \$a1, 7 # bottom row is row 7 (Y_SPACE - 1)

li \$a2, X_SPACE # X_SPACE (4)

add \$a3, \$zero, \$t4

jal calculateArrayAddress

add \$t0, \$v0, \$zero # copy returned value to t register

pop (\$ra)

lb \$t1, empty

sb \$t1, 0(\$t0)

add \$t4, \$t4, \$t2

sb \$t4, playerXpos

add \$t0, \$t0, \$t2

lb \$t1, player

sb \$t1, 0(\$t0)

movePlayer_exit:

jr \$ra

shootPlayer:

```
    push    ($ra)
    la      $a0, gridSize
    li      $a1, 7 # row above player is row 6 (Y_SPACE - 2)
    li      $a2, X_SPACE # X_SPACE (4)
    lb      $t3, playerXpos
    add     $a3, $zero, $t3
    jal     calculateArrayAddress
    addi    $t0, $v0, 0 # copy returned value to t register
    pop     ($ra)

    lb      $t2, -4($t0) # if trying to shoot an enemy in front, replace with empty space
    lb      $t1, enemy
    beq     $t2, $t1, shootPlayer_front

    lb      $t1, bullet
    j       shootPlayer_spawn
shootPlayer_front:
    lb      $t1, dead # killed an enemy

    lb      $t4, deadCount
    addi    $t4, $t4, 1 # increment enemies killed count
    sb      $t4, deadCount

    addi    $t0, $t0, -4
    push    ($t0) # place address of dead onto stack for clearDead

shootPlayer_spawn:
    sb      $t1, -4($t0) # either a bullet or dead enemy
    push    ($ra)
    jal     sound_shoot # shooting sound effect
    pop     ($ra)

shootPlayer_exit:
    jr      $ra
```

moveBullet:

scanning top to bottom

```
    li      $t0, 0 # i = 0
moveBullet_whileColumn:
    bge     $t0, X_SPACE, moveBullet_doneColumn # i < X_SPACE (4)

    li      $t1, 0 # j = 0
```

moveBullet_whileRow:

bge \$t1, Y_SPACE, moveBullet_doneRow # j < Y_SPACE (8)

push (\$ra)

la \$a0, gridSize

add \$a1, \$zero, \$t1 # row

li \$a2, X_SPACE # X_SPACE (4) columns

add \$a3, \$zero, \$t0 # column

jal calculateArrayAddress

add \$t2, \$zero, \$v0 # copy returned value to t register

pop (\$ra)

lb \$t4, bullet

lb \$t3, 0(\$t2)

bne \$t3, \$t4, moveBullet_nextRow # branch if not a bullet

moveBullet_bulletFound:

lb \$t4, empty # delete current bullet

sb \$t4, 0(\$t2)

empty) beq \$t1, 0, moveBullet_nextRow # special case, top row (replace with

lb \$t3, -4(\$t2) # check spot above bullet

lb \$t4, enemy

beq \$t3, \$t4, moveBullet_enemyHit

lb \$t4, empty

beq \$t3, \$t4, moveBullet_empty

moveBullet_enemyHit:

lb \$t4, dead # killed an enemy, replace with dead sprite

lb \$t6, deadCount

addi \$t6, \$t6, 1 # increment enemies dead count

sb \$t6, deadCount

addi \$t5, \$t2, -4

push (\$t5) # place address of dead onto stack for clearDead

j moveBullet_shift

moveBullet_empty:

lb \$t4, bullet

```

        j      moveBullet_shift
moveBullet_shift:
    sb      $t4, -4($t2)    # replace above spot with bullet or dead sprite

        j      moveBullet_nextRow

        moveBullet_nextRow:
            addi   $t1, $t1, 1    # ++j
        j      moveBullet_whileRow
moveBullet_doneRow:
    addi   $t0, $t0, 1    # ++i
        j      moveBullet_whileColumn
moveBullet_doneColumn:
    jr      $ra

```

pauseGame:

```

    li      $t0, 2    # set to game state 2 (pause)
    sb      $t0, gameState

    # draw pause screen
    la      $a0, frameBuffer
    la      $a1, file_frameBuffer_M11
    lw      $a2, file_width_M11
    lw      $a3, file_height_M11
    jal     printSprite

    j      gamePause

```

spawnEnemy:

```

    lw      $t0, lfsr      # get a random number
    andi    $t0, $t0, 7    # take only numbers 0-7

    # 50% chance to spawn an enemy, enemy spawns at the "top" of gridSpace

    bgt     $t0, 3, spawnEnemy_exit    # if (random_number <= 3)
    push    ($ra)    # save $ra to stack
    la      $a0, gridSpace
    li      $a1, 0    # top row is row 0
    li      $a2, X_SPACE    # X_SPACE (4), row is 0 anyways
    add     $a3, $zero, $t0    # spawn in a random column
    jal     calculateArrayAddress
    add     $t0, $v0, $zero    # copy returned value to t register
    pop     ($ra)    # restore $ra register

```

```

lb    $t1, enemy    # place an enemy at the array address
sb    $t1, 0($t0)

```

```

spawnEnemy_exit:
jr    $ra

```

moveEnemy:

start from the "bottom left" of the grid, iterate upwards

this will prevent moving an enemy that was already moved

```

li    $t0, 0    # i = 0

```

moveEnemy_whileColumn:

```

bge    $t0, X_SPACE, moveEnemy_doneColumn    # i < X_SPACE (4)

```

```

li    $t1, 7    # j = Y_SPACE - 1 (7)

```

moveEnemy_whileRow:

```

blt    $t1, 0, moveEnemy_doneRow    # j >= 0

```

```

push   ($ra)

```

```

la     $a0, gridSpace

```

```

add    $a1, $zero, $t1    # row

```

```

li     $a2, X_SPACE    # X_SPACE (4) columns

```

```

add    $a3, $zero, $t0    # column

```

```

jal    calculateArrayAddress

```

```

add    $t2, $zero, $v0    # copy returned value to t register

```

```

pop    ($ra)

```

```

beq    $t1, 0, moveEnemy_atTop    # special case, top row (replace with

```

empty)

```

lb     $t3, -4($t2)    # copy what is one row above (-4 bytes)

```

```

# is it an enemy?

```

```

lb     $t4, enemy

```

```

beq    $t3, $t4, moveEnemy_enemy# branch if it isn't

```

```

# is it an empty space?

```

```

lb     $t4, empty

```

```

beq    $t3, $t4, moveEnemy_empty

```

```

j      moveEnemy_nextRow

```

```

moveEnemy_enemy: # if it is an 'enemy'

```

```

beq    $t1, 7, moveEnemy_playerHit    # if enemy reached

```

the bottom, hit player

```

lb     $t4, 0($t2)    # check what is at the spot already

```

```

# is there a bullet?

```

```

lb    $t5, bullet
beq   $t4, $t5, moveEnemy_bulletHit
# is there a player?
lb    $t5, player
beq   $t4, $t5, moveEnemy_playerHit

# if there wasn't either then
sb    $t3, 0($t2)    # place at array address
j     moveEnemy_nextRow

```

moveEnemy_empty:

```

lb    $t4, 0($t2)    # check what is at the spot already
# is there a bullet?
lb    $t5, bullet
beq   $t4, $t5, moveEnemy_nextRow
# is there a player?
lb    $t5, player
beq   $t4, $t5, moveEnemy_nextRow

# if it wasn't either then
sb    $t3, 0($t2)    # place at array address
j     moveEnemy_nextRow

```

moveEnemy_bulletHit:

```

lb    $t4, dead
sb    $t4, 0($t2)    # enemy killed (removes both enemy and

```

bullet)

```

lb    $t7, deadCount
addi  $t7, $t7, 1    # increment dead count
sb    $t7, deadCount

```

```

push  ($t2)    # save address of where dead enemy was placed

```

for clearDead

```

j     moveEnemy_nextRow

```

moveEnemy_playerHit:

```

lb    $t6, playerHP
addi  $t6, $t6, -1    # player hp -1, hp will get checked in

```

updateState

```

sb    $t6, playerHP

```

```

##### play a sound

```

```

        push  ($ra)
        jal   sound_hurt
        pop   ($ra)

        j     moveEnemy_nextRow

moveEnemy_atTop:
        lb    $t4, empty
        sb    $t4, 0($t2)    # an enemy can't move to the top row

        moveEnemy_nextRow:
        addi  $t1, $t1, -1    # --j
        j     moveEnemy_whileRow
moveEnemy_doneRow:

        addi  $t0, $t0, 1     # ++i
        j     moveEnemy_whileColumn
moveEnemy_doneColumn:
        jr    $ra

```

clearDead:

```

        lb    $t2, deadCount
clearDead_loop:
        beq   $t2, 0, clearDead_exit # iterate until all dead enemies are cleared

        # top of stack should be an address of a dead enemy sprite
        pop   ($t0)    # $t0 is the address of a dead enemy sprite
        lb    $t1, empty
        sb    $t1, 0($t0)    # replace dead enemy sprite with empty
        ##### play a sound
        push  ($ra)
        jal   sound_hit
        pop   ($ra)

        addi  $t2, $t2, -1    # remove an enemy from death count

        lb    $t1, score
        addi  $t1, $t1, 1     # increase score count
        sb    $t1, score

        j     clearDead_loop
clearDead_exit:
        sb    $t2, deadCount

```

```

        jr      $ra

print:
##### replace with bitmap
        li      $t0, 0  # i = 0
        print_whileRow:
            bge   $t0, Y_SPACE, print_doneRow    # while (i < X_SPACE)

            li    $t1, 0  # j = 0
            print_whileColumn:
                bge   $t1, X_SPACE, print_doneColumn  # while (j < Y_SPACE)

                push ($ra)
                la    $a0, gridSpace
                add   $a1, $zero, $t0                # row iterates after column, row-major
                li    $a2, X_SPACE
                add   $a3, $zero, $t1
                jal   calculateArrayAddress          # calls calculateArrayAddress
subroutine
                pop  ($ra)

                lb    $a0, 0($v0)  # PRINT TO TERMINAL
                li    $v0, 11
                syscall

                addi  $t1, $t1, 1    # ++j
                j     print_whileColumn
            print_doneColumn:

            newline

            addi  $t0, $t0, 1    # ++i
            j     print_whileRow
        print_doneRow:

        newline

        jr      $ra

printBitmap:
# will scan through entire game array and draw the bitmap accordingly (inefficient, will redraw
EVERY pixel)

```

```

la    $t0, gridSpace# base array address
li    $t1, 0 # gridSpace index, all 32 elements, 0 - 31
li    $t7, 0x000aa0aa # $t7 the color to be used
push  ($ra)

printBitmap_while:
    bge    $t1, 32, printBitmap_exit    # index < 31

    # we translate the gridSpace address to the corresponding bitmap address
    sll    $t2, $t1, 2    # turn 1 byte to 4 bytes (size of bitmap elements)
    la     $t3, framebuffer    # top left corner of bitmap display
    add    $t3, $t3, $t2    # corresponding address on bitmap

    # addi $t3, $t3, 4    # add the space between sprites

    add    $t4, $t0, $t1    # corresponding address on gridSpace
    lb     $t6, 0($t4)

    # where to start drawing sprites
    lw     $t2, gameScreenLocation
    sll    $t2, $t2, 2
    li     $t4, 4
    div    $t1, $t4    # hi - x coord, lo - y coord
    mflo   $t4
    mfhi   $t3

    sll    $t3, $t3, 6    # x-coord * 32 * 4
    sll    $t4, $t4, 13    # y-coord * 256 * 4

    add    $t3, $t3, $t4
    add    $t3, $t3, $t2

    # switch case - compare to current element
    lb     $t5, player
    beq    $t6, $t5, printBitmap_player
    lb     $t5, enemy
    beq    $t6, $t5, printBitmap_enemy
    lb     $t5, empty
    beq    $t6, $t5, printBitmap_empty
    lb     $t5, dead
    beq    $t6, $t5, printBitmap_dead
    lb     $t5, bullet
    beq    $t6, $t5, printBitmap_bullet

```

```

j      printBitmap_draw
# determine what will be drawn by looking at grid space
printBitmap_player:
    push    ($ra)
    la      $a0, framebuffer($t3)
    la      $a1, file_frameBuffer_M3
    lw      $a2, file_width_M3
    lw      $a3, file_height_M3
    jal     printSprite
    pop     ($ra)

j      printBitmap_draw
printBitmap_enemy:
    push    ($ra)
    la      $a0, framebuffer($t3)
    la      $a1, file_frameBuffer_M5
    lw      $a2, file_width_M5
    lw      $a3, file_height_M5
    jal     printSprite
    pop     ($ra)

j      printBitmap_draw
printBitmap_empty:
    push    ($ra)
    la      $a0, framebuffer($t3)
    la      $a1, file_frameBuffer_M7
    lw      $a2, file_width_M7
    lw      $a3, file_height_M7
    jal     printSprite
    pop     ($ra)

j      printBitmap_draw
printBitmap_dead:
    push    ($ra)
    la      $a0, framebuffer($t3)
    la      $a1, file_frameBuffer_M6
    lw      $a2, file_width_M6
    lw      $a3, file_height_M6
    jal     printSprite
    pop     ($ra)

j      printBitmap_draw
printBitmap_bullet:
    push    ($ra)

```

```

        la    $a0, framebuffer($t3)
        la    $a1, file_frameBuffer_M4
        lw    $a2, file_width_M4
        lw    $a3, file_height_M4
        jal   printSprite
        pop   ($ra)

        j     printBitmap_draw

```

```

printBitmap_draw:
    addi    $t1, $t1, 1    # ++index
    j       printBitmap_while
printBitmap_exit:
    pop     ($ra)
    jr      $ra

```

printSprite:

\$a0 - coordinate address to start drawing from (la \$a0, framebuffer(\$some register))
 # \$a1 - sprite to draw (la \$a1, file_frameBuffer_M#) # depending on sprite
 # \$a2 - sprite width (lw \$a2, file_width_M#) same # as above
 # \$a3 - sprite height (lw \$a3, file_height_M#) same # as above

\$t0 - framebuffer address, \$t1 - gridSpace index [don't mess with here]

\$t4 - actual spot where drawing occurs
 # \$t5 - row index, \$t6 - column index
 # \$t7 - color of sprite at row/column
 # \$v0 - address of sprite at sprite coordinates

```

    li      $t5, 0    # y = 0
printSprite_row:
    bge     $t5, $a3, printSprite_doneRow

    li      $t6, 0    # x = 0
printSprite_column:
    bge     $t6, $a2, printSprite_doneColumn

```

the sprites data

```

    mul     $t4, $t5, $a2    # v0 = row * spriteWidth
    add     $t4, $t4, $t6    # v0 = (row * spriteWidth) + column
    sll     $t4, $t4, 2      # v0 = 4*((row * spriteWidth) + column)
    add     $t4, $t4, $a1    # v0 = base address + 4*((row * spriteWidth) + column))
    lw      $t7, 0($t4)     # color of sprite at this address

```

```

# store onto the bitmap
lw      $t4, framebufferWidth
mul      $t4, $t5, $t4    # v0 = row * number_of_columns
add      $t4, $t4, $t6    # v0 = (row * number_of_columns) + column
sll      $t4, $t4, 2      # v0 = 4*((row * number_of_columns) + column)
add      $t4, $t4, $a0    # v0 = base address + 4*((row * number_of_columns) + column))
sw      $t7, 0($t4)      # when the bitmap gets updated

```

```

addi     $t6, $t6, 1
j        printSprite_column
printSprite_doneColumn:

```

```

addi     $t5, $t5, 1
j        printSprite_row
printSprite_doneRow:
printSprite_exit:
jr       $ra

```

calculateArrayAddress:

a0 - base address, \$a1 - row, \$a2 - nColumns, \$a3 - column

v0 - address returned

```

mul      $v0, $a1, $a2    # v0 = row * number_of_columns
add      $v0, $v0, $a3    # v0 = (row * number_of_columns) + column
add      $v0, $v0, $a0    # v0 = base address + ((row * number_of_columns) + column))

```

```

jr       $ra

```

lfsr32:

/* HW4 code written by me

a0 = n (use seed when initializing, 0 otherwise)

t0 - lfsr, t1 - bit, t2 - intermediate taps

v0 = returned random number

```

bne      $a0, $zero, lfsr32_nNotZero # if (n == 0)

```

```

lw       $t0, lfsr        # load lfsr from memory

```

```

srl      $t1, $t0, 0      # lfsr >> 0, tap 32
srl      $t2, $t0, 10     # lfsr >> 10, tap 22
xor      $t1, $t1, $t2    # (tap 32) ^ (tap 22)

```

```

srl      $t2, $t0, 30     # lfsr >> 30, tap 2
xor      $t1, $t1, $t2    # ((tap 32) ^ (tap 22)) ^ (tap 2)

```

```

srl    $t2, $t0, 31    # lfsr >> 31, tap 1
xor    $t1, $t1, $t2    # ((tap 32) ^ (tap 22) ^ (tap 2)) ^ (tap 1)

andi   $t1, $t1, 1      # bit = (taps xor) & 1

sll    $t1, $t1, 31     # bit << 31
srl    $t0, $t0, 1      # lfsr >> 1
or     $t0, $t0, $t1     # lfsr = (lfsr >> 1) | (bit << 31)

sw     $t0, lfsr         # store new lfsr to memory
add    $v0, $t0, $zero   # return lfsr

j      lfsr32_exit

lfsr32_nNotZero:                                     # else

        sw     $a0, lfsr         # lfsr = n
        j      lfsr32_exit

lfsr32_exit:
jr     $ra

```

initialTime:

```

li     $v0, 30
# initial time call
syscall
# $a0 - low order 32 bits, $a1 - high order 32 bits
sw     $a1, startTimeUpper
sw     $a0, startTimeLower
# store this initial time to startTime to be compared against
jr     $ra

```

currentTime:

```

li     $v0, 30
# current time call
syscall
# $a0 - low order 32 bits, $a1 - high order 32 bits
lw     $t1, startTimeUpper
lw     $t0, startTimeLower
# subtract startTime from currentTime
sub    $t1, $a1, $t1
sub    $t0, $a0, $t0
# left with time elapsed between initial and current in $t1, $t0

```

30 seconds = 30000 ms - lower order bits (0111 0101 0011 0000) - 0x7530

only need to compare \$t0, \$t1 is free to use

li \$t1, 0x7530 # 30 seconds in hex

bge \$t0, \$t1, increaseDifficulty # if (difference >= 30)

j noIncrease

increaseDifficulty:

lb \$t0, difficulty

beq \$t0, 1, noIncrease

increased difficulty is lowering the delay between enemy turns

lower "difficulty" number = harder

addi \$t0, \$t0, -1

sb \$t0, difficulty

get a new initial time (resets 30s timer)

push (\$ra)

jal initialTime

pop (\$ra)

noIncrease:

jr \$ra

sound_hit:

the sound that plays when you get a hit

li \$a0, 72

li \$a1, -1

li \$a2, 112

li \$a3, 100

li \$v0, 31

syscall

jr \$ra

sound_hurt:

the sound that plays when you are hit

li \$a0, 64

li \$a1, -1

li \$a2, 120

li \$a3, 100

li \$v0, 31

syscall

jr \$ra

sound_shoot:

```

# the sound that plays when you shoot
    li    $a0, 64
    li    $a1, -1
    li    $a2, 127    # gunshot sound
    li    $a3, 100
    li    $v0, 31
    syscall

    jr    $ra

```

sound_titleToGamePlay:

```

# the sound that plays when you switch from title to gameplay
    li    $a0, 69
    li    $a1, 250
    li    $a2, 112
    li    $a3, 100
    li    $v0, 31
    syscall

    li    $v0, 33
    li    $a0, 66
    syscall

    jr    $ra

```

sound_gameOver:

```

    li    $a0, 66
    li    $a1, 250
    li    $a2, 112
    li    $a3, 100
    li    $v0, 31
    syscall

    li    $v0, 33
    li    $a0, 69
    syscall

    jr    $ra

```

```

#include <stdio.h>
#include <stdlib.h>

#define X_SPACE 10
#define Y_SPACE 10

void makeSpace(char space[][Y_SPACE]);
void drawSpace(const char space[][Y_SPACE]);
void initPlayer(char space[][Y_SPACE]);
void initEnemies(char space[][Y_SPACE]);

void moveLogic(char space[][Y_SPACE], char key);
void movePlayer(char space[][Y_SPACE], int x, int y, char key);
void movePlayerProjectile(char space[][Y_SPACE], int x, int y);
void moveEnemy(char space[][Y_SPACE], int x, int y);
void removeEnemy(char space[][Y_SPACE], int x, int y);
void moveEnemyProjectile(char space[][Y_SPACE], int x, int y);

int main(void) {

    char gridSpace[X_SPACE][Y_SPACE]; // game grid
    makeSpace(gridSpace);
    // drawSpace(gridSpace);

    initPlayer(gridSpace);
    initEnemies(gridSpace);

    int win = 1;
    int frame = 0;

    char empty = ' ';

    char keyPressed = '0';

    drawSpace(gridSpace);
    while (win != 0) {
        scanf("%c", &keyPressed);
        //playerControl(player, gridSpace, keyPressed);
        moveLogic(gridSpace, keyPressed);
        drawSpace(gridSpace);
        printf("[GRID MARKED AS DOTS FOR VISUAL PURPOSES]\n");

        frame += 1;
    }
    printf("you lost\n");

    return 0;
}

```

```

}

void makeSpace(char space[][Y_SPACE]) { // initialize grid
    size_t i = 0;
    while (i < X_SPACE) {
        size_t j = 0;
        while (j < Y_SPACE) {
            space[i][j] = '.';
            ++j;
        }
        ++i;
    }
}

void drawSpace(const char space[][Y_SPACE]) { // print grid
    size_t j = 0;
    while (j < Y_SPACE) {
        size_t i = 0;
        while (i < X_SPACE) {
            printf("%c ", space[i][j]);
            ++i;
        }
        printf("\n");
        ++j;
    }
}

void collision(char space[][Y_SPACE], int x, int y) {
    switch(space[x][y]) {

    }
}

// check collision between player and enemies

void initPlayer(char space[][Y_SPACE]) {
    // player
    char player = 'P';
    char playerProjectile = '^';

    // player position on grid
    space[X_SPACE / 2][Y_SPACE - 1] = player;
}

void initEnemies(char space[][Y_SPACE]) {
    // enemy
    char enemy1 = 'V';
    char enemy2 = 'W';

```

```

char enemyDead = 'x';
char enemyProjectile = 'v';

// place enemies on grid
int enemyCount = 4;

size_t i = 0;
while (i < enemyCount) {
    space[(2*i + 1) % (X_SPACE - 1)][((2*i + 1) / X_SPACE)*2] =
enemy1;
    ++i;
}

}

void moveLogic(char space[][Y_SPACE], char key) { // acts bottom to top
    int i = X_SPACE - 1;
    while (i >= 0) {
        int j = Y_SPACE - 1;
        while (j >= 0) {
            switch(space[i][j]) {
                case 'P':
                    movePlayer(space, i, j, key);
                    break;
                case '^':
                    movePlayerProjectile(space, i, j);
                    break;
                case 'V':
                    moveEnemy(space, i, j);
                    break;
                case 'W':
                    moveEnemy(space, i, j);
                    break;
                case 'x':
                    removeEnemy(space, i, j);
                    break;
                case 'v':
                    moveEnemyProjectile(space, i, j);
                    break;
                default:
                    break;
            }
            --j;
        }
        --i;
    }
}

```

```

void movePlayer(char space[][Y_SPACE], int x, int y, char key) {
    switch(key) {
        case 'a':
            if (x > 0) {
                space[x][y] = '.';
                x -= 1;
            }
            break;

        case 'd':
            if (x < X_SPACE - 1) {
                space[x][y] = '.';
                x += 1;
            }
            break;

        case 'w':
            space[x][y - 1] = '^';
            break;

        default:
            break;
    }

    space[x][y] = 'P';
}

```

```

void movePlayerProjectile(char space[][Y_SPACE], int x, int y) {
    space[x][y] = '.'; // remove from initial position
    y -= 1; // go up one space
    if (x < 0 || x > X_SPACE) // guard clauses
        return;
    if (y < 0 || y > Y_SPACE)
        return;
    space[x][y] = '^';
}

```

```

void moveEnemy(char space[][Y_SPACE], int x, int y) {
    space[x][y] = '.';
}

```

```

void removeEnemy(char space[][Y_SPACE], int x, int y) {
    space[x][y] = '.';
}

```

```

void moveEnemyProjectile(char space[][Y_SPACE], int x, int y) {

```

```
space[x][y] = '.'; // remove from initial position
y += 1; // go down one space
if (x < 0 || x > X_SPACE) // guard clauses
    return;
if (y < 0 || y > Y_SPACE)
    return;
space[x][y] = 'v';
```

Any supplementary files that would be needed to reproduce your test results. This should include anything you used to create and test your project and may include supporting .h files, supporting C code, spreadsheets, Matlab scripts, data files, etc.

I just included every file I used for this project in this google drive folder including:

- Mars (Mars4_5.jar) and the Mars on windows workaround (Mars4_5.bat)

- The main program .asm file (projectFinal16.asm)

- All the needed .rgba files (there are 9 and they end in .rgba, they are REQUIRED)

- And the super unfinished C code (ProjectDraft-RhiloSotto.c)

Running a Mars instance from the folder containing all the .rgba files and the .asm file should allow the files to be read in the main program.

https://drive.google.com/drive/folders/1IEuVcd6SuL2NTic9qPr3kjr1IYTyVrfS?usp=drive_link

Credits for the pixel art and transcribing the music go to Danelle Verdadero.

(I used <https://convertio.co/rgba-converter/> to turn the pngs she made into rgba files)